SKILL

User Input

```
$ARGUMENTS
```

You **MUST** consider the user input before proceeding (if not empty).

Pre-Execution Checks

Check for extension hooks (before constitution update): - Check if `.specify/extensions.yml` exists in the project root. - If it exists, read it and look for entries under the `hooks.before_constitution` key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions: - If the hook has no `condition` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation - When constructing slash commands from hook command names, replace dots (`.`) with hyphens (`-`). For example, `speckit.git.commit` → `/skill:speckit-git-commit`. - For each executable hook, output the following based on its `optional` flag: - **Optional hook** (`optional: true`): ``` ## Extension Hooks

```
**Optional Pre-Hook**: {extension}
Command: `/{command}`
Description: {description}

Prompt: {prompt}
To execute: `/{command}`
```
```

- **Mandatory hook** (`optional: false`):

  ```
  ## Extension Hooks

  Automatic Pre-Hook: {extension}
  Executing: `/{command}`
  ```
  ```

```
EXECUTE_COMMAND: {command}

Wait for the result of the hook command before proceeding to
the Outline.
```

After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.

- If no hooks are registered or `.specify/extensions.yml` does not exist, skip silently

Outline

You are updating the project constitution at `.specify/memory/constitution.md`. This file is a TEMPLATE containing placeholder tokens in square brackets (e.g. `[PROJECT_NAME]`, `[PRINCIPLE_1_NAME]`). Your job is to (a) collect/derive concrete values, (b) fill the template precisely, and (c) propagate any amendments across dependent artifacts.

Note: If `.specify/memory/constitution.md` does not exist yet, it should have been initialized from `.specify/templates/constitution-template.md` during project setup. If it's missing, copy the template first.

Follow this execution flow:

1. Load the existing constitution at `.specify/memory/constitution.md`.
    - Identify every placeholder token of the form `[ALL_CAPS_IDENTIFIER]`. **IMPORTANT**: The user might require less or more principles than the ones used in the template. If a number is specified, respect that - follow the general template. You will update the doc accordingly.
2. Collect/derive values for placeholders:
    - If user input (conversation) supplies a value, use it.
    - Otherwise infer from existing repo context (README, docs, prior constitution versions if embedded).
    - For governance dates: `RATIFICATION_DATE` is the original adoption date (if unknown ask or mark TODO), `LAST_AMENDED_DATE` is today if changes are made, otherwise keep previous.

    - `CONSTITUTION_VERSION` must increment according to semantic versioning rules:
        - MAJOR: Backward incompatible governance/principle removals or redefinitions.
        - MINOR: New principle/section added or materially expanded guidance.
        - PATCH: Clarifications, wording, typo fixes, non-semantic refinements.
    - If version bump type ambiguous, propose reasoning before finalizing.

3. Draft the updated constitution content:

    - Replace every placeholder with concrete text (no bracketed tokens left except intentionally retained template slots that the project has chosen not to define yet—explicitly justify any left).
    - Preserve heading hierarchy and comments can be removed once replaced unless they still add clarifying guidance.
    - Ensure each Principle section: succinct name line, paragraph (or bullet list) capturing non-negotiable rules, explicit rationale if not obvious.
    - Ensure Governance section lists amendment procedure, versioning policy, and compliance review expectations.

4. Consistency propagation checklist (convert prior checklist into active validations):

    - Read `.specify/templates/plan-template.md` and ensure any "Constitution Check" or rules align with updated principles.
    - Read `.specify/templates/spec-template.md` for scope/requirements alignment—update if constitution adds/removes mandatory sections or constraints.
    - Read `.specify/templates/tasks-template.md` and ensure task categorization reflects new or removed principle-driven task types (e.g., observability, versioning, testing discipline).
    - Read each installed Spec Kit command file for your agent (including this one) — named `speckit.*` or `speckit-*` (dot or hyphen depending on the agent), or laid out as `speckit-<name>/SKILL.md` for skills-based integrations, e.g. in `.github/agents/`, `.github/skills/`, `.claude/skills/`, or your agent's equivalent commands directory — to verify no outdated references (CLAUDE-only or other agent-specific names) remain when generic guidance is required.
    - Read any runtime guidance docs (e.g., `README.md`, `docs/quickstart.md`, or agent-specific guidance files if present). Update references to principles changed.

5. Produce a Sync Impact Report (prepend as an HTML comment at top of the constitution file after update):

    - Version change: old → new

    - List of modified principles (old title → new title if renamed)
    - Added sections
    - Removed sections
    - Templates requiring updates (updated / ⚠ pending) with file paths
    - Follow-up TODOs if any placeholders intentionally deferred.

6. Validation before final output:

    - No remaining unexplained bracket tokens.
    - Version line matches report.
    - Dates ISO format YYYY-MM-DD.
    - Principles are declarative, testable, and free of vague language (“should” → replace with MUST/SHOULD rationale where appropriate).

7. Write the completed constitution back to `.specify/memory/constitution.md` (overwrite).

8. Output a final summary to the user with:

    - New version and bump rationale.
    - Any files flagged for manual follow-up.
    - Suggested commit message (e.g., `docs: amend constitution to vX.Y.Z (principle additions + governance update)`).

Formatting & Style Requirements:

- Use Markdown headings exactly as in the template (do not demote/promote levels).
- Wrap long rationale lines to keep readability (<100 chars ideally) but do not hard enforce with awkward breaks.
- Keep a single blank line between sections.
- Avoid trailing whitespace.

If the user supplies partial updates (e.g., only one principle revision), still perform validation and version decision steps.

If critical info missing (e.g., ratification date truly unknown), insert `TODO(<FIELD_NAME>): explanation` and include in the Sync Impact Report under deferred items.

Do not create a new template; always operate on the existing `.specify/memory/constitution.md` file.

Post-Execution Checks

Check for extension hooks (after constitution update): Check if `.specify/extensions.yml` exists in the project root. - If it exists, read it and look for entries under the `hooks.after_constitution` key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter

out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions: - If the hook has no `condition` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation - When constructing slash commands from hook command names, replace dots (`.`) with hyphens (`-`). For example, `speckit.git.commit` → `/skill:speckit-git-commit`. - For each executable hook, output the following based on its `optional` flag: - **Optional hook** (`optional: true`): ``` ## Extension Hooks

```
**Optional Hook**: {extension}
Command: `/{command}`
Description: {description}

Prompt: {prompt}
To execute: `/{command}`
```
```

- **Mandatory hook** (`optional: false`):

  ```
  ## Extension Hooks

  **Automatic Hook**: {extension}
  Executing: `/{command}`
  EXECUTE_COMMAND: {command}
  ```

  After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.

- If no hooks are registered or `.specify/extensions.yml` does not exist, skip silently
  ```